01・自己紹介

セツ シュントウ（Xue Juntao）

学年	大学4年 2027年9月卒業予定
専攻・学部	立教大学 文学部史学科 古代地中海史ゼミ

＊原・米国バージニア工科大進学予定（コロナ禍により進路変更）

出身地	中国上海
補足	米国工学系高校出身 （ロボットの図面制作を中心に活動）

エンジニアを志したきっかけ 📚

複雑構造への関心 × 個人創作経験 → システム設計志向

- 📐統合設計
- 説明可能なシステム
- プロダクト志向
- プロトタイピング
- 構造設計

02・スキル・技術スタック

Rapid Prototyping / プロダクト開発

⛏️開発経験:2.5年+

🚀アイデアを短期間で
プロトタイプとして立ち上げ、
検証可能な形に落とし込めます

個人開発・フルスタック
（動的・静的Webサービス運営）
システム／プロトタイプ設計～実装 (Spiral)
ローカル／クラウド間の状態同期
Pythonによる自動化・GUIツール
小規模ツールチェーン構築 など

🛠️🛠️高抽象度の設計・試作

Custom DSL・命令系の設計～実装
メタ制御系CLIの設計～実装
権限制御・状態遷移・履歴構造の設計
自然言語入力を前提とした操作系の設計

主な技術

プロダクト設計 ・情報設計

構造整理・ワークフロー設計・ UX設計中心一貫
🍓Smultron, 🪨Obsidian, 🦋Whimsical
Clip Studio Paint, Jyupter Notebook
Figma, Adobe Suite

AI開発環境・プロトタイピング

自然言語を用いた操作設計や、AIを含む
ワークフローの検証・プロトタイピング
Claude, ChatGPT, Cursor, Replit,
Anthropic / OpenAI API

開発環境（プロトタイプ実装）

TypeScript / JavaScript, Rust(basic),
Python(basic), C(basic), React,
Node.js / Express, Flask, REST APIs,
WebSocket PostgreSQL, Firebase

Development base: Linux, Bash
Version Control: Git

03・今後のキャリアビジョン

志望・興味分野

・AIワークフロー設計・要件定義・開発プロセス改善
・業務改善につながる社内ツールの提案・設計・実装
・課題を見つけ、全体像を踏まえて構造化すること 🪜
・人が使いやすい、負担を下げるインタラクティブなソフトウェア
・設計を通じた職種間コミュニケーションの橋渡し

志望・興味業界

複雑な課題を整理し、プロダクトとして価値に変えられる会社💪

目指しているキャリア像

改善や効率向上につながる課題を見つけ、企画から実装まで一貫して価値に変えられるビルダーになりたい🔨

企業に対する期待

フロントエンド・バックエンド・設計を過度に分断せず、プロダクトの目的やユーザー価値、業務の流れまで含めて一貫して考え、価値づくりにつなげていける環境

04・代表プロジェクトの設計史

Spiral（スパイラル）

履歴をベースに、
対話と文書構造を扱うシステム

プロジェクト概要

種別	リサーチプロトタイピング / プロダクト志向のフルスタック開発
役割	Founder / Architect / Developer
実装	React, Node.js, PostgreSQL, AI API
URL	spiral.ooo (SaaS prototype) core.rec.ooo (OSS kernel) entaticspiral.com (Documentation) ux.rec.ooo (UX showcases)

⚙️技術的な実装ポイント（低層）

- append-only のイベント履歴設計
  Append-only event history design
- event DAG による因果関係管理
  Causal relationship management using an event DAG
- 意味のある「最近文脈」の導出
  Derivation of meaningful recent context
- 自然言語と DSL を組み合わせた操作系設計
  Interaction model design combining natural language and DSL*複雑な履歴や操作理由を追えるようにして、ユーザーが状態変化を理解しやすくするため
- 状態変化の理由を追跡できる説明可能性の設計
  Explainability design for tracing the reasons behind state changes
- 対話型 UX と構造操作の統合
  Integration of conversational UX and structural manipulation

🧠 工夫した点（高層）

- Spiral 本体となるシステム構造の定義
- RWX ベースの権限・操作モデルの設計
- 文書・対話・操作を一つの workspace として扱う UX 設計
- 自然言語による制御を前提にした interaction surface の封装
- モジュール / 権限 / identity をまたぐ一貫したプロダクト設計

→複雑な概念やシステム構造を、一つの使いやすく理解しやすいインターフェースに統合

初期段階の Spiral OS

05 · 開発の流れの補足

Spiral（スパイラル）

履歴をベースに、
対話と文書構造を扱うシステム

プロジェクト概要

種別	リサーチプロトタイピング / プロダクト志向のフルスタック開発
役割	Founder / Architect / Developer
実装	React, Node.js, PostgreSQL, AI API
URL	spiral.ooo (SaaS prototype) core.rec.ooo (OSS kernel) entaticspiral.com (Documentation) ux.rec.ooo (UX showcases)

🐧 RWX Interactionの開発の流れの例

自然言語による設計定義と実装推進

1. 要件整理 / 定義
複雑な内部構造を扱いながらも
操作感はシンプルに保つ方針を設定

↓

2. 画面設計
Desktop / Fragment Editor の分離による
閲覧・編集・実行体験の整理

↓

3. 権限設計
Read / Write / Execute の分離による
表示と操作の境界整理

↓

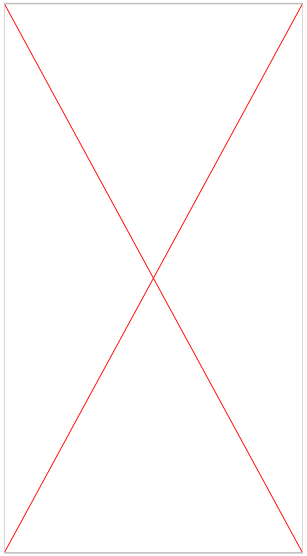
4. データ構造設計
文書タイプ・モジュール定義のメタデータ化と
表示・実行判断基盤の整理

↓

5. AI / バックエンド責務分離
文脈解釈や自動補完は AI
権限判定や状態更新は通常ロジックに分離

→

6. 体験への統合
内部は複雑でも、外からは理解しやすく操作しやすい
workspace として見えるようにまとめた



PWA化したspiral.ooo (prototype playground)

06・このシステムを使うと何が変わるか

🌟コア特性
- トレーサブルな設計
- 説明可能な状態遷移
- AIとの安定したインタラクション

Before

いつものシステムでは…

- ユーザー操作 → 状態変化が見えない
- ログが分散している
- なぜ起きたか追跡しにくい

After

Spiralだと、

- ユーザー操作 → 構造化された fragment として記録
- すべての変更が履歴としてつながる
- システムの挙動を追跡・説明できる

❗なぜ重要か

デバッグや原因特定がしやすい
チーム間での理解が揃いやすい
複雑なワークフローの曖昧さを減らす

つまり、

→ 複雑なシステムを「観測可能で制御可能」な状態に変える

07 · Thank You

🔍 AIからSpiralを知ろう
設計思想・構造・実装範囲を短時間で把握したい場合は、
spiral.ooo／peek をご利用ください。
要約をAIに貼り付け、自由に質問できます。

spiral.ooo (SaaS prototype)
core.rec.ooo (OSS kernel)
entaticspiral.com (Documentation)
ux.rec.ooo (UX showcases)

スライドの閲覧・ダウンロードはこちら
https://s.rec.ooo/career/engineer-1-on-1/